DYNAMIC ENSEMBLE LEARNING FOR CREDIT CARD FRAUD DETECTION - CRISP-DM PROCESS REPORTs

BUSINESS UNDERSTANDING

The widespread adoption of electronic payment systems has significantly boosted global e-commerce activity and digital financial transactions. While this transformation has enhanced convenience for consumers and operational efficiency for businesses, it has also exposed financial systems to increasing threats from credit card fraud. In 2021 alone, global losses due to credit card fraud were estimated at $32.2 billion and are projected to rise to $49.32 billion by 2030. These growing threats have made credit card fraud detection (CCFD) a critical research area, with a direct impact on consumer trust, regulatory compliance, and economic stability. Recent studies have demonstrated that machine learning (ML) and deep learning (DL) models can substantially improve fraud detection rates.

Traditional ML classifiers such as logistic regression, decision trees, random forests, AdaBoost, and gradient-boosting variants (e.g., XGBoost, CatBoost) have demonstrated strong performance on historical transaction data}. Meanwhile, deep models—LSTM, GRU, CNNs, and Transformers—have proven effective in capturing temporal, spatial, and contextual dependencies. Ensemble learning methods like bagging, boosting, and stacking have also been employed to enhance robustness.

Despite these advancements, the use of Dynamic Ensemble Selection (DES) in CCFD remains largely unexplored. DES dynamically select the most competent classifiers from an ensemble for each test instance based on local accuracy estimates, offering a flexible and context-sensitive alternative to static ensembles. Unlike static ensembles, DES methods tailor the prediction process to individual instances by evaluating local competence. The rationale behind these techniques is that not every classifier in the pool is an expert in classifying all unknown samples; rather, each base classifier is an expert in a specific local region of the feature space. Furthermore, while resampling methods such as SMOTE, SMOTE-ENN, and RUS have been extensively used to mitigate class imbalance in fraud datasets, their integration with DES approaches has received limited attention.

In this work, we propose a novel approach to credit card fraud detection by integrating DES methods with resampling techniques such as RandomUndersampling and ClusteringCentroids, in addition with class-weighted learning. By framing fraud detection as a local competence problem, we enhance the adaptability and robustness of the model compared to traditional static ensembles.

DATA UNDERSTANDING: *Data Description Report*

- **Location:** The original dataset is available at https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud
- **Method used to acquire:** Download the dataset using the `kagglehub` library, which provides a simple way to interact with Kaggle resources.
- **Format:** csv file
- **Quantity:** Rows (284807) - Columns (31)
- **Identities of the fields**: The dataset contains transactions made by credit cards in September 2013 by European cardholders. This dataset presents transactions that occurred in two day. It contains only numerical input variables which are the result of a PCA transformation. Due to confidentiality issues, we cannot provide the original features and more background information. Features V1, V2, … V28 are the principal components obtained with PCA, the only features which have not been transformed with PCA are 'Time' and 'Amount'. Feature 'Time' contains the seconds elapsed between each transaction and the first transaction in the dataset.
- **Surfaces features:** Amount, Time, PCs (V1 … V28), Class
  - ***Quantitative Variables***
    - Discrete: -
    - Continuous: Amount, Time, PCs (V1 … V28)
  - ***Categorical Variables***
    - Nominal: Class (0=Legit / 1=Fraud)
    - Ordinal: -

DATA UNDERSTANDING: *Data Quality Report*

- **Missing values Analysis**
  - ***Column-Level Analysis*:** Checked the presence of missing values in each of the columns to understand if some imputation was required. There are 0 columns with missing values.
  - ***Row-Level Analysis:*** Checked the presence of rows with more than 50% of missing values among the columns. There are 0 rows with more than 50% of missing values among the columns.

- **Class Balance:** The dataset presents a very high imbalance, as already know for frauds classification problems: Legit-0 (284315 – 99.827%) / Fraud (492 – 0.173%). In the next steps the data imbalance should be handled by using undersampling or oversampling techniques or by adopting models that can handle this very high imbalance within the dataset.

- **Duplicated Row:** The dataset contains 1081/284807 (0.380%) duplicated rows. In particular, it contains 773 duplicated patterns distributed as follows (611 different patterns repeated 2 times, 66 different patterns repeated 3 times, 81 different patterns repeated 4 times, 10 different patterns repeated 5 times, 1 pattern repeated 6 times, 2 different patterns repeated 9 times, 2 different patterns repeated 18 times). Additionally, we check duplicated both in terms of all features + Class and only looking to the features; in both case we have 1081/284807 (0.380%).
  If we leave duplicates in the dataset we may end up in:
  - Bias in the model - The classifier will "see" repeated evidence of the same row. This gives unfair influence to those examples during training.
  - Risk of overfitting - Especially with a small number of fraud cases, repeating exact Class 1 samples inflates the model's confidence.
  - Invalidation of cross-validation - Duplicates might leak into both training and validation folds. This leads to optimistically biased performance metrics (especially precision/recall).

  Based on these considerations, one solution could be of dropping duplicated rows, keeping each of them just one time.

DATA UNDERSTANDING: *Data Exploration Report*

- **Class Balance:** The dataset presents a very high imbalance: Legit-0 (284315 – 99.827%) / Fraud-1 (492 – 0.173%).

- **Time Analysis:** Time contains the seconds elapsed between each transaction and the first transaction in the dataset
  - The skewness value of approximately -0.036 is very close to zero. This indicates that the distribution is highly symmetrical.
  - The mean (94813.86) is slightly greater than the median (84692.00). This small difference confirms the slight negative skew (mean should be less than the median for negative skew) and indicates that the data has a slightly heavier tail on the positive (right) side than on the negative (left) side, despite the skewness value being negative.
  - The time values range from 0.00 to 172792.00. The middle 50% of the data spans from 54201.50 (25%) to 139320.50 (75%).
  - The standard deviation (47488.14) is relatively large compared to the mean, but the symmetrical nature suggests the spread is consistent with a normal-like distribution, and the min/max values appear reasonable for the overall range.
  - For Legit transactions, we can observe a repetitive wave pattern with two large peaks. This suggests transaction volume follows a diurnal (daily) cycle: possibly office hours and nighttime dips, repeated over two days. This gives us confidence that time reflects real-world temporal behavior.
  - For Fraud transactions, we have much lower volume, but the key insight is that fraud attempts are not uniformly distributed. We can see clusters/spikes in specific time windows. This suggests frauds may be targeted in bursts, likely by bots or coordinated attacks.

- **Transaction Amount Distribution**
  - The distribution of Amount is highly skewed right (approximately 16.98). This means the vast majority of the transactions have very small amounts, and the long, thin tail of the distribution extends far out to the right due to a small number of extremely large transactions. These large transactions are acting as significant outliers.
  - The mean (88.47) is far greater than the median (22.00). The mean is being pulled significantly upwards by the extremely large values in the right tail, while the median remains a robust measure of the typical transaction amount.
  - The standard deviation (250.4) is extremely large compared to the mean. This indicates a long right tail, which is a common indicator of a highly skewed distribution and the presence of extreme outliers.
  - Most transactions are small (75% of transactions are ≤ 77.51).
  - There are zero-amount transactions. This may represent failed/incomplete attempts, errors, or may indicate fraud tests (e.g., verifying a card).
  - Logarithmic Transformation [log(x + 1)] is applied, end up into a more symmetric and usable form. This transformation has successfully normalized the Amount feature, pulling the extreme right tail back towards the center of the distribution. Log transform stabilizes variance, compresses outliers, and makes fraud/legit distributions visually and statistically manageable.

- **PCA Feature Analysis**
  - We don't have information on how the PCA was computed. Thus, we perform preliminary checks knowing how the PCA works and what is expected:
    - PCA must be performed on centered data (mean=0).
      - All the PCs (V1 … V28) have mean ~ 0, suggesting that data have been centered before applying the PCA (also the kde plots show that the distribution are centered around 0);
    - In PCA we have a variance reduction from the first to the last principal direction.
      - We compute the standard deviation and we can confirm that there is a variance reduction, as expected.
    - Median and quartile are symmetric distributed and centered.

        - Correlation matrix shows that all the PCs have 0 correlation, as expected, since PCA applies orthogonal projections.
    - We use the cumulative explained variance to find a cut-off and find the number of PCs to retain at least 85% (V1 … V17), 90% (V1 … V20) and 95% (V1 … V23) of the total variance.
    - Comparing the kde for each PCs between the two classes emerged:
        - V4, V10, V11, V12, V14, V17 are highly promising candidates for model discrimination and maybe feature selection.
        - V2, V3, V5, V6, V8, V13, V16, V18, V22 may not separate well alone but could help in interaction with others.
        - V1, V7, V9, V19, V20, V21, V23, V28, V15, V26 are probably noise or redundant for classification.

- **Features Relevance to Target class**
    - ***Pearson Correlation***
        - PCA features V11–V17 carry strong linear separability.
        - V11, V12, V14, V17, V10 showed strongest (negative) correlation with fraud class.
        - LogAmount, Hour, Time showed weak or no linear correlation.
        - Amount close to 0 correlation.

    - ***Mutual Information***
        - No feature has extremely high MI (> 0.01), which is expected due to high class imbalance and feature obfuscation (PCA).
        - V17, V14, V12, V11, V10, V27 showed highest MI — top fraud indicators.
        - LogAmount, Hour — Moderate MI (more relevant than many PCA features).
        - Amount (raw) — Less informative than LogAmount.
        - Many PCA features (e.g., V1, V4, V3, V8) — Carry small but non-zero signal.

DATA PREPARATION: *Data Sampling Report*

- **Sampling Approach**
  - ***keep_all_minority_with_ratio:*** To address the class imbalance in the 284315 record dataset [Legit-0 (284315 – 99.827%) / Fraud-1 (492 – 0.173%)], a sub-sampling strategy was employed to create a smaller, computationally efficient, and representative sample. The strategy, *"keep_all_minority_with_ratio"*, involved retaining all instances of the minority 'Fraud' class and performing a proportional random under-sampling of the 'Legit' class at a 50:1 ratio. This process yielded a new sample of 25092 records [Legit-0 (24600 – 98.039%) / Fraud (492 – 1.961 %)].
    - Running configuration:
      - *policy='keep_all_minority'*
      - *seed=42*
      - *n_rows=None*
      - *frac=None*
      - *ratio=50*

- **Post Sampling Validation**
  - ***A rigorous two-part statistical validation*** was then conducted to ensure this new sample remained representative of the original population. The validation confirms that the sub-sampled dataset is a statistically robust and highly representative proxy for the original data.

  - ***Validation of Individual Feature Distributions (Univariate Analysis):***
    - We first tested whether the distribution of each individual feature in the sample (n=25092) was statistically identical to its distribution in the population (N=284315).
    - Initial checks using descriptive statistics (mean, median, quartiles, std. dev) and visualizations (histograms, violin plots) showed a near-perfect visual overlap for all features.
    - A two-sample Kolmogorov-Smirnov (K-S) test was formally conducted for all features.

  - ***Validation of Feature Relationships (Multivariate Analysis):***
    - We then tested whether the complex relationships between features were preserved during sampling.
    - We computed the Pearson correlation matrix for the original population and the new sample.
    - An "error matrix" was calculated by taking the absolute difference between the two correlation matrices.
    - The analysis of this error matrix was excellent.
      - Mean Error: The average error across all feature relationships was approximately 0.0105 (1.05%).
      - Max Error: The single "worst-case" error observed for any feature pair was approximately 0.102 (10.2%).

  The K-S tests confirm that individual features retained their statistical properties. The correlation analysis confirms that the relationships between features were also preserved with extremely high fidelity.

The initial sampling has been performed in order to have a small, but still representative, dataset to be used for modeling due to limited computational resources. However, the sampled dataset is still heavily unbalanced. **In this work, we explore different undersampling techniques to rebalance the dataset before to feed the machine learning algorithm.**

SELECTED PARAMETERS:

- RandomUnderSampler : random_state=42, sampling_strategy=0.10
- ClusterCentroids : random_state=42, sampling_strategy=0.10

In order to avoid data leakage and to rely just on the training set, the resampling is performed only on the training set (after Data Construction and Feature Selection) in each round of the cross validation procedure.

DATA PREPARATION: *Data Cleaning Report*

In the Data Quality phase we assess the following quality problems:

- **Missing values Analysis**
  - ***Column-Level Analysis*****:** no-problems
  - ***Row-Level Analysis:*** no-problems

- **Class Balance:** The initial dataset presents a very high imbalance, as already know for frauds classification problems: Legit-0 (284315 – 99.827%) / Fraud-1 (492 – 0.173%). After the data sampling phase (performed to reduce the dataset size for computational limits), the new dataset still have imbalance [Legit-0 (24600 – 98.039%) / Fraud-1 (492 – 1.961%)].
  To avoid leakage and bias, we choose to use oversampling techniques to rebalance the dataset, which will be implemented during the training phase only on the training dataset (and not on the validation or test datasets).

- **Duplicated Row:** The dataset contains 1081/284807 (0.380%) duplicated rows in the initial dataset. To avoid bias in the model – since the classifier will "see" repeated evidence of the same row – we choose to remove these duplicated occurrences.
  The sampled dataset contains 25092 rows [Legit-0 (24600 – 98.039%) / Fraud-1 (492 – 1.961%)]
  We found 25/25092 (0.10%) of duplicated rows. After dropping the cleaned dataset presents the following statistics: 25067 rows [Legit-0 (24594 – 98.113%) / Fraud-1 (473 – 16.13%)].

DATA PREPARATION: *Feature Selection Report*

Feature selection is another important step of the Data Preparation phase because redundant or irrelevant features can have significant effects on the quality of the results of the analysis method.
As reported in (*Saeys Y, Inza I, Larrañaga P. A review of feature selection techniques in bioinformatics. Bioinformatics. 2007 Oct 1;23(19):2507-17*), an advisable practice is to pre-reduce the search space using a univariate filter method, and only then apply a wrapper or embedded methods, hence fitting the computation time to the available resources.
In this work, we use the following method to define the optimal set of features:

1. **Filter method:** Univariate feature selection works by selecting the best features based on univariate statistical tests.
   - ***SelectKBest*** removes all but the k highest scoring features. We can choose the statistical test that we want to use to identify the best features.
     - ***f_classif*** ANOVA F-value between label/feature for classification tasks.
     - ***mutual_info_classif*** Mutual information for a discrete target. Mutual information between two random variables is a non-negative value, which measures the dependency between the variables. It is equal to zero if and only if two random variables are independent, and higher values mean higher dependency.

   SELECTED PARAMETERS: f_classif, k tuned in the search space [20, 22, 24, 26, 28, "all"]

In order to avoid data leakage and to rely just on the training set, the feature selection is performed only on the training set in each round of the cross validation procedure.

DATA PREPARATION: *Data Construction Report*

- **Amount:** In the Data Exploration phase we assess that the Amount feature is highly right skewed (with very long tail). Moreover, there are zero-amount transactions. This may represent failed/incomplete attempts, errors, or may indicate fraud tests (e.g., verifying a card).
  - ***Logarithmic Transformation*** [log(x + 1)] is applied, end up into a more symmetric and usable form. This transformation has successfully normalized the Amount feature, pulling the extreme right tail back towards the center of the distribution. Log transform stabilizes variance, compresses outliers, and makes fraud/legit distributions visually and statistically manageable. This transformation can be directly applied with no-leakage risk.
  - ***StandardScaler*** is then applied on Amount_log1p to transform the distribution into a Standard distribution with 0 mean and unit variance.
    In order to avoid data leakage and to rely just on the training set, the StandardScaler is performed only on the training set in each round of the cross validation procedure.
    This transformation is always fit only on the training dataset and then the computed statistics are used to transform validation and test data. In this way we can avoid data leakage from the test set.

- **Time:** Time contains the seconds elapsed between each transaction and the first transaction in the dataset. We chose to transform this feature to better represent cyclicity.
  - ***Trigonometric features*** are used to encode Time periodic feature using a sine and cosine transformation with the matching period. Each ordinal time feature is transformed into 2 features that together encode equivalent information in a non-monotonic way, and more importantly without any jump between the first and the last value of the periodic range.
  - ***Period*** Since we have two days of data in seconds, we have 60*60*24 sec = 86400 sec in one day. We use period = 86400 to represent these two days of data in terms of sine and cosine on the trigonometric circle.

- **PCA Feature V1 … V28:** As already checked in previous steps, these are the PCs already computed (with all the required adjustment computed). No additional transformation are required.

DATA PREPARATION: *Data Integration Report*

NOTHING TO REPORT

DATA PREPARATION: *Data Format Report*

NOTHING TO REPORT

MODELING
NOTHING TO REPORT

EVALUATION
NOTHING TO REPORT

DEPLOYMENT
NOTHING TO REPORT
